Контракт фронт ↔ бэк

Голосовой ассистент для задач и встреч · хакатон Gorkycode · предложение фронта, v1.3 · 20 октября 2026

Статус	Предложение. Фронт будет разрабатываться на моках по этому документу
Утверждают	Лёша — бэкенд, Маша — ИИ-модуль (P-49)
Обозначения	обяз. поле есть всегда · необяз. может быть `null` · резерв под допы, в MVP не нужно · P-xx — номер решения · v1.3 — появилось в этой версии

- **Один эндпоинт на всё**: текст, распознанный голос и нажатия кнопок идут в `POST /api/chat` (P-50).
- **Ответ — всегда одна структура**: тип, текст, записи, кнопки и пара служебных полей. Тип только подсказывает фронту оформление (P-38).
- **LLM только разбирает запрос**. Даты, проверки и действия — код бэка. Фразы ответа — шаблоны, а не генерация (P-58).
- **Кнопки присылает бэк**. Чего бэк не умеет — той кнопки просто нет, фронт переделывать не нужно (P-28).
- **Распознавание речи — в браузере**. Бэк получает уже текст (P-44).
- **Нужны HTTPS и CORS** на бэке, иначе браузер не пустит запросы (раздел 3).
- **Ассистента зовут Stevens** — в тексте латиницей, в озвучке «Стивенс». Все его фразы берутся из шаблонов раздела 9 (P-120, P-121, P-141).
- **Вход и песочница — в MVP** v1.3 . Первый экран — вход или регистрация, рядом кнопка «Попробовать без регистрации»: песочница со своей копией демо-данных. Раздел 14 (P-179).

1. Кто за что отвечает

ИИ-модуль (LLM)

- Текст запроса → JSON: намерение и поля
- Уверенность в разборе
- Не считает даты сам, не пишет в БД, не сочиняет ответ

Бэкенд

- Превращает «завтра», «в пятницу» в дату по `client_time` и `timezone`
- Проверки: нехватка данных, прошлое, совпадение встреч
- Запись и поиск в БД
- Собирает текст ответа из шаблона, решает, какие кнопки показать

Фронтенд

- Запись голоса, распознавание (в браузере)
- Отправка, показ ответа по его типу
- Сам вычисляет только «прошедшая или нет» (P-53)
- Сам показывает ошибки сети и таймаута (P-51)

2. Эндпоинты

Метод и путь	Зачем	Когда
`GET /api/health`	Жив ли сервер. Фронт вызывает при открытии страницы; пока нет ответа — показывает «Просыпаюсь...» (P-46). Ответ: `{"status": "ok"}`. Без заголовка режима	MVP
`POST /api/chat`	Любое сообщение ассистенту: текст, голос, кнопка. Разделы 4–7. С заголовком режима (раздел 14)	MVP

`POST /api/auth/register` v1.3	Регистрация: `{login, password}` → `{token, user}`. Аккаунт создаётся пустым. Раздел 14	MVP
`POST /api/auth/login` v1.3	Вход: `{login, password}` → `{token, user}`. Раздел 14	MVP
`POST /api/auth/logout` v1.3	Выход: токен перестаёт действовать → `204`	MVP
`GET /api/auth/me` v1.3	Проверка токена при открытии страницы → `{user}` или `401`	MVP
`POST /api/sandbox/reset` v1.3	Сброс песочницы: демо-данные этого `X-Client-Id` копируются заново → `204`	MVP
`POST /api/stt`	Аудио → текст. Только если распознавание в браузере не взлетит (P-44)	резерв
`GET /api/records`	`?from=YYYY-MM-DD&to=YYYY-MM-DD` → `{"records": Record[]}`. Для календарной панели. С заголовком режима	резерв

3. Общие правила

- **Кодировка** UTF-8, тело запросов и ответов — JSON. Имена полей и значения-перечисления — на английском (P-48), тексты для пользователя — на русском.
- **Дата записи** — `"YYYY-MM-DD"`, **время** — `"HH:MM"` в 24-часовом формате. Это местное время пользователя, без часового пояса, как в CSV.
- **Моменты** (`client_time`, `created_at`) — ISO 8601 со смещением: `"2026-10-18T14:05:00+03:00"`.
- **«Сегодня»** — реальная дата (P-54). Бэк берёт её из `client_time` запроса, а не из часов сервера: сервер может стоять в другом поясе.
- **HTTPS обязателен** для фронта и бэка. Микрофон в браузере работает только на HTTPS, а со страницы HTTPS браузер не пускает запросы на HTTP.
- **CORS**: фронт и бэк будут на разных адресах. Бэку нужно разрешить домен фронта (и `http://localhost:5173` для разработки), методы `GET`, `POST` и заголовки `Content-Type`, `Authorization`, `X-Client-Id` v1.3. Куки не используем.
- **Коды ответа.** Любой ответ ассистента, включая «не понял», — `200`. Неверный запрос — `422`. Недействительный токен — `401` v1.3, фронт тогда выводит на экран входа. Внутренний сбой — `500`. Всё, что не 2xx, фронт показывает как ошибку со своей кнопкой «Повторить» (P-34, P-51).
- **Таймаут** на стороне фронта — 25 секунд (P-47). Если LLM бывает медленнее — скажите, обсудим.

4. Запрос: `POST /api/chat`

v1.3 Кроме тела, каждый запрос несёт заголовок режима: `Authorization: Bearer <token>` в аккаунте или `X-Client-Id: <uuid>` в песочнице. Подробно — раздел 14.

Поле	Тип	Смысл
`conversation_id` обяз.	`string` (UUID)	Идентификатор диалога. Генерирует фронт при первом открытии. Нужен, чтобы понять, что «в 10:00» — ответ на вопрос «На какое время?» (P-45)
`text` обяз.	`string`	Текст пользователя, не длиннее 300 символов; длиннее — ответ `422` (P-92, P-132). Для кнопки — её надпись или `send.text`
`source` обяз.	`"voice" | "text" | "button"`	Откуда пришло сообщение. На логику не влияет, полезно для логов и отладки
`action` необяз.	`Action | null`	Заполнено, только если нажата кнопка-действие (раздел 6). Тогда бэк ориентируется на `action`, а не на `text`
`client_time` обяз.	`string` (ISO)	Текущее время устройства пользователя
`timezone` обяз.	`string` (IANA)	Например, `"Europe/Moscow"`

```
{
  "conversation_id": "7f3c2a9e-1b4d-4e8a-9c21-5d0f6a7b8c90",
  "text": "Поставь встречу с Иваном завтра",
  "source": "voice",
  "action": null,
  "client_time": "2026-10-18T14:05:00+03:00",
  "timezone": "Europe/Moscow"
}
```

5. Ответ

Поле	Тип	Смысл
`message_id` обяз.	`string`	Уникальный id ответа
`kind` обяз.	`string`	Тип ответа, каталог в разделе 7. Незнакомый тип фронт всё равно покажет по остальным полям (P-38)
`text` обяз.	`string`	Фраза ассистента для показа. Собирается из шаблона раздела 9 (P-58, P-122)
`speech` необяз.	`string | null`	Короткая фраза для озвучки: список дел вслух не читаем. `null` — озвучивать `text`. Сама озвучка — доп (P-39, P-06)
`records` обяз.	`Record[]`	Карточки записей, раздел 6. Может быть пустым
`nearest` необяз.	`Record | null`	Только для пустого `list`: ближайшая будущая запись (P-17)
`total` необяз.	`number | null`	Для `list` — число записей периода (в `records` приходят все). Для `found` — сколько всего совпадений, даже если в `records` меньше (P-42, P-130)
`awaiting_reply` обяз.	`boolean`	`true`, если ассистент задал вопрос и ждёт ответа. Отвечать необязательно (P-33)
`buttons` обяз.	`Button[]`	Кнопки под ответом, раздел 6. Может быть пустым

6. Вложенные структуры

Record — запись

Поле	Тип	Смысл
`id` необяз.	`number | null`	`null` — запись ещё не сохранена (черновик в `confirm` и `conflict`)
`type` необяз.	`"task" | "reminder" | "meeting" | null`	Три типа различаем (Р-12). `null` — тип неизвестен, например у записей из CSV (О-01)
`text` обяз.	`string`	Содержание. Тексты из CSV — как есть (Р-56)
`date` необяз.	`string | null`	`"YYYY-MM-DD"`. `null` — задача без срока
`time` необяз.	`string | null`	`"HH:MM"`. `null` — без времени
`date_kind` обяз.	`"exact" | "deadline" | "none"`	`exact` — в этот момент; `deadline` — «до этой даты», фронт пишет «до вс, 25 окт»; `none` — без срока (Р-40)
`status` необяз.	`"open" | "done" | null`	Для задач и напоминаний. У встреч — `null`
`created_at` необяз.	`string | null` (ISO)	Дата создания. Для записей из CSV — время импорта или `null`
`marks` обяз.	`string[]`	`"new"` — только что создана; `"draft"` — ещё не сохранена; `"conflict"` — совпадает по времени с новой встречей. Обычно пусто
`highlight` необяз.	`"date" | "time" | "text" | null`	Какое поле выделить, если спрашивали только его («Когда была встреча?» → `date`)
`remind_at` резерв	`string | null` (ISO)	Когда прислать пуш. Появится вместе с уведомлениями (Д7, Р-40)

Button — кнопка

Поле	Тип	Смысл
`label` обяз.	`string`	Надпись. После нажатия появляется в ленте как сообщение пользователя (Р-55)
`send` обяз.	`{ text } | { action }`	Что отправить: либо текст (быстрый ответ «10:00»), либо действие. Ровно одно из двух
`style` необяз.	`"primary" | "secondary"`	По умолчанию `primary`. «Отменить» — `secondary`

Нажатие кнопки фронт отправляет обычным `POST /api/chat` с `source: "button"`: `text` = `send.text` или `label`, `action` = `send.action`. После нового сообщения пользователя кнопки старых ответов становятся неактивными (Р-33).

Action — действия кнопок

id	Параметр	Что делает бэк	Когда
`cancel`	—	Отменяет текущее незавершённое действие, отвечает `cancelled` (Р-20)	MVP
`confirm_yes`	—	Выполняет действие из `confirm`	MVP
`confirm_no`	—	Не выполняет; просит сказать иначе (`clarify`)	MVP
`keep_both`	—	Сохраняет новую встречу на занятое время	MVP
`change_new_time`	—	Спрашивает новое время для новой встречи (`clarify`)	MVP
`move_existing`	`record_id`	Спрашивает новое время для существующей встречи. Нужно изменение записей; не умеет — не присылает кнопку	MVP*
`undo`	`record_id`	Отменяет только что созданную запись (Д10, Р-41)	резерв
`complete`	`record_id`	Отмечает выполненной (Д3)	резерв
`delete`	`record_id`	Удаляет запись (Д2)	резерв

Формат: `{"id": "move_existing", "record_id": 688}`. * — если бэк успеет изменение записей.

7. Типы ответов (`kind`)

kind	Когда	Что заполнено	Ждёт
`created`	Создана задача, напоминание или встреча	`records`: созданная запись с `marks: ["new"]`	нет
`list`	Планы за период: «Что у меня сегодня?»	Все записи периода по возрастанию даты и времени, `total`. Пусто → `nearest` (Р-17, Р-130)	нет
`found`	Результаты поиска, включая прошлое	`records` — до 50 лучших, в порядке бэка; `total`. На вопрос «когда?» — текст называет дату, у записей `highlight` (Р-42, Р-43)	нет
`clarify`	Не хватает данных или дата в прошлом	`text` — вопрос; `buttons` — подсказки (необязательно) + «Отменить»	да
`confirm`	Высокий риск, что понял неправильно (Р-19)	`records`: черновик «как понял»; кнопки Да / Нет	да
`conflict`	Новая встреча на время, где уже есть встреча (Р-26)	`records`: черновик (`draft`) + существующие (`conflict`); кнопки по правилу 6, раздел 8	да
`cancelled`	Нажата «Отменить»	Только `text`	нет
`not_understood`	Не удалось разобрать запрос	`buttons`: 2–3 примера фраз (`send.text`)	нет
`off_topic`	Запрос не про планы (Р-32)	«Я умею только...» + примеры в `buttons`	нет
`help`	«Что ты умеешь?» (Р-31)	Описание + примеры в `buttons`	нет
`thanks`	Пользователь благодарит (Р-124)	Только `text`	нет

`error`	Бэк не смог выполнить (LLM недоступна, сбой БД)	Только `text`. «Повторить» фронт добавит сам	нет
`updated` `deleted` `completed`	Изменение, удаление, выполнение (Д1–Д3)	`records`: запись в новом состоянии	резерв

8. Правила бэка, которые видит пользователь

1. **Задача без даты** сохраняется без срока, без переспрашивания: `date_kind: "none"`. «До конца недели», «до пятницы» → `deadline` (Р-40).
2. **Напоминание без времени** → `clarify`, шаблон **`clarify_reminder_time`** (Р-18).
3. **Встреча без времени** → `clarify`, шаблон **`clarify_meeting_time`** (пример из Т3).
4. **Напоминание без дня и без времени** («Напомни про отчёт», «Напомни на следующей неделе...») → `clarify`, шаблон **`clarify_reminder_when`** (Р-135).
5. **Есть время, но нет дня** («Созвон с партнёром в 14:00») → `clarify`, шаблон **`clarify_day`**. Сами день не подставляем (Р-133).
6. **Встреча без участника и без темы** («Поставь встречу на пятницу в 15») → `clarify`, шаблон **`clarify_meeting_person`**. Если есть хотя бы участник или тема («Демо проекта в четверг в 16:00») — создаём сразу (Р-134).
7. **Дата или время в прошлом** → `clarify`, шаблон **`clarify_date_past`** (Р-36).
8. **Уверенный разбор — сразу выполняем**, без «всё верно?». `confirm` — только при высоком риске ошибки (Р-19).
9. **Совпадение встреч.** Сравниваются только встречи со встречами, по точному времени начала. Задачи и напоминания не конфликтуют (Р-26, Р-27, Р-30). Кнопки:
   - на это время ровно одна встреча → «Другое время для новой», «Перенести существующую», «Оставить обе», «Отменить»;
   - две и больше → то же без «Перенести существующую» (Р-28).
10. **Пустой период** → `list` с пустым `records` и ближайшей будущей записью в `nearest`, шаблон **`list_empty_nearest`**. Если спрашивали про один тип («Есть ли встречи?»), `nearest` — ближайшая запись этого типа (Р-17).
11. **Поиск** — по смыслу, а не по точному совпадению (Т3), включая прошлые записи. До 50 лучших и `total` (Р-42). По анализу аналитика команды, в данных 387 упоминаний людей и 95 компаний, и все — в творительном падеже («Отправить договор Иваном», «с Альфой»). Запрос «Иван» должен находить «Иваном»: нужна нормализация словоформ, например лемматизация (Р-136).
12. **Незавершённое уточнение.** Если следующее сообщение — не ответ на вопрос, обрабатываем его как новый запрос, а старое уточнение забываем (Р-33).
13. **Тексты ответов — шаблоны** из раздела 9, с подстановкой данных (Р-58, Р-122).
14. **Благодарность** («спасибо», «благодарю», «спасибо, Стивенс», «спасибо, Stevens») → `thanks`. Если в том же сообщении есть просьба, выполняем просьбу (Р-124).
15. **Даты в тексте** совпадают с датами в `records`: текст собирается из тех же данных, что и карточка.
16. **Не обещаем того, чего нет.** Пока нет пуш-уведомлений (Д7), ассистент не говорит «напомню», а говорит «записал напоминание». С появлением пушей шаблоны **`created_reminder`** поменяем (Р-128).

9. Шаблоны ответов

Тон — «тень Stevens»: вежливый, точный и сдержанный ассистент с лёгкими штрихами дворецкого, без пародии на дворецкого (Р-121). Тексты ниже утверждены (Р-122).

Как пользоваться

1. Для ответа выбираете ID по ситуации, затем **случайный** из его вариантов. Подряд один и тот же вариант лучше не повторять.
2. Подставляете переменные (таблица ниже) в `text`. Если у шаблона указана «Озвучка» — подставляете переменные и кладёте её в `speech`, иначе `speech: null`.
3. Если фраза начинается с переменной, первая буква значения делается заглавной. В шаблонах это обозначено заглавной буквой в имени: `{Period}`, `{Type}`, `{When}` (P-123).
4. Текст записи `{text}` всегда стоит в кавычках-ёлочках и подставляется как есть, без смены регистра: так не ломаются «CRM» и имена (P-125).
5. Если для ответа нет подходящего ID, присылайте `clarify_generic` или `not_understood`, но не сочиняйте новую фразу.

Переменные

Переменная	Что подставлять в `text`	В `speech`
`{text}`	`Record.text` как есть	так же
`{date}`	«сегодня», «завтра», «вчера»; иначе «пн, 19 окт»; если год не текущий — «пт, 5 мар 2027» (P-112)	«сегодня» / «завтра» / «вчера» или «19 октября» (+ «2027 года»), **без дня недели и сокращений** (P-127)
`{time}`	«HH:MM», с ведущим нулём: «08:00»	так же
`{when}`	По `date_kind`: `exact` → «{date} в {time}» или просто «{date}», если времени нет; `deadline` → «до {date}»; `none` → «без срока»	то же, с речевой формой даты
`{period}`	Период запроса как обстоятельство: «на сегодня», «завтра», «на этой неделе», «на следующей неделе», «с 19 по 25 окт»	то же, месяц словом
`{type}`	«задача» | «напоминание» | «встреча»	так же
`{types_gen}`	«задач» | «напоминаний» | «встреч»	так же
`{count:ф1|ф2|ф3}`	Число цифрами + нужная форма: «3 дела», «11 дел», «21 запись»	так же

Выбор формы для `{count:ф1|ф2|ф3}` (P-126): ф1 — если число оканчивается на 1, кроме 11 (1, 21, 101); ф2 — если оканчивается на 2–4, кроме 12–14 (2, 23, 104); ф3 — все остальные (5–20, 25, 111, 0).

Формы типов для `<формы типа>` в **`list_some`** и **`list_type_yes`**: задача|задачи|задач · напоминание|напоминания|напоминаний · встреча|встречи|встреч.

Шаблоны

ID	Когда	Варианты и озвучка
`created_task` kind: created	Создана задача с датой (`date_kind: exact`)	1. Записал задачу на {when}: «{text}». 2. Готово: «{text}» — {when}. 3. Задача «{text}» стоит на {when}. Озвучка: Записал на {when}.
`created_task_deadline` kind: created	Создана задача со сроком (`deadline`)	1. Записал: «{text}», срок — до {date}. 2. Готово: «{text}» — до {date}. 3. Срок — до {date}. Задачу «{text}» записал. Озвучка: Записал, срок — до {date}.

`created_task_nodate` kind: created	Создана задача без срока (`none`)	1. Записал задачу без срока: «{text}». 2. Готово: «{text}», без срока. 3. Задачу «{text}» сохранил, без срока. Озвучка: Записал, без срока.
`created_reminder` kind: created	Создано напоминание	1. Записал напоминание на {when}: «{text}». 2. Записал: {when} — «{text}». 3. Напоминание на {when} сохранил: «{text}». Озвучка: Напоминание — {when}.
`created_meeting` kind: created	Создана встреча	1. Встреча записана: «{text}», {when}. 2. Готово: {when} — «{text}». 3. Добавил встречу «{text}» на {when}. Озвучка: Встреча — {when}.
`list_some` kind: list	Планы за период. Если спрашивали про один тип («Какие задачи на завтра?») — формы этого типа, см. ниже	1. {Period} — {count:дело|дела|дел}: 2. {Period} запланировано {count:дело|дела|дел}: 3. Вот ваши планы {period}: {count:дело|дела|дел}. Озвучка: {Period} — {count:дело|дела|дел}.
`list_type_yes` kind: list	«Есть ли у меня встречи / задачи / напоминания?» — да	1. Да, {count:<формы типа>}: 2. Есть, {count:<формы типа>}: 3. Да, у вас {count:<формы типа>}: Озвучка: Да, {count:<формы типа>}.
`list_empty_nearest` kind: list	За период пусто, `nearest` есть	1. {Period} планов нет. Ближайшее — {when}: «{text}». 2. {Period} ничего не запланировано. Следующее — «{text}», {when}. 3. {Period} свободно. Ближайшее дело — {when}: «{text}». Озвучка: {Period} планов нет. Ближайшее — {when}.
`list_empty_none` kind: list	За период пусто, впереди тоже ничего (`nearest: null`)	1. {Period} планов нет, и дальше пока пусто. 2. {Period} пусто, ближайших планов тоже нет. 3. Расписание свободно. Скажите, что записать.
`list_type_empty` kind: list	Про один тип — пусто. Если `nearest` нет, вторая фраза не нужна	1. {Period} {types_gen} нет. Ближайшее — {when}: «{text}». 2. {Period} {types_gen} не запланировано. Следующее — «{text}», {when}. 3. {Period} у вас нет {types_gen}. Ближайшее — {when}: «{text}». Озвучка: {Period} {types_gen} нет.
`found_some` kind: found	Найдено {count} (больше одной)	1. Нашёл {count:запись|записи|записей}: 2. Вот что нашлось — {count:запись|записи|записей}: 3. Нашлось {count:совпадение|совпадения|совпадений}: Озвучка: Нашёл {count:запись|записи|записей}.
`found_one` kind: found	Найдена одна запись	1. Нашёл одну запись: 2. Есть одна: «{text}», {when}. 3. Нашлась одна запись: «{text}». Озвучка: Нашёл одну запись.
`found_none` kind: found	Ничего не найдено	1. Ничего не нашёл. Попробуйте другое слово или дату. 2. Боюсь, ничего нет. Уточните название или период. 3. Совпадений нет. Попробуйте искать иначе.

found_when_last kind: found	«Когда была…?» — называем самую позднюю прошедшую из найденных. Она идёт первой в `records`	1. В последний раз — {when}: «{text}». 2. Последний раз — {when}. 3. Последняя запись — {when}: «{text}». Озвучка: В последний раз — {when}.
found_when_next kind: found	«Когда у меня…?» — называем ближайшую будущую. Она идёт первой в `records`	1. Ближайший раз — {when}: «{text}». 2. В следующий раз — {when}. 3. Ближайшая запись — {when}: «{text}». Озвучка: В следующий раз — {when}.
clarify_meeting_time kind: clarify	Встреча без времени	1. Во сколько встреча? 2. Позвольте уточнить: в какое время? 3. Укажите, пожалуйста, время встречи.
clarify_reminder_time kind: clarify	Напоминание без времени	1. Во сколько напомнить? 2. В какое время напомнить? 3. Укажите время напоминания.
clarify_reminder_when kind: clarify	Напоминание без дня и без времени	1. Когда напомнить: день и время? 2. Позвольте уточнить: когда напомнить? 3. Назовите день и время напоминания.
clarify_day kind: clarify	Есть время, но нет дня (встреча, напоминание)	1. На какой день? 2. Какого числа? 3. Уточните, пожалуйста, день.
clarify_meeting_person kind: clarify	Встреча без участника и без темы	1. С кем встреча? 2. С кем или о чём будет встреча? 3. Уточните, пожалуйста: с кем встреча?
clarify_date_past kind: clarify	Дата или время уже прошли	1. Это уже в прошлом. На когда записать? 2. {When} уже позади. На когда записать? 3. Боюсь, это время уже прошло. Выберите другое.
clarify_generic kind: clarify	Не хватает данных (общий случай)	1. Не хватает деталей: что и на когда? 2. Позвольте уточнить: что именно и когда? 3. Чуть больше подробностей — и запишу.
confirm kind: confirm	«Правильно ли я понял»	1. Правильно понимаю: {type} «{text}», {when}? 2. {Type} «{text}», {when} — верно? 3. Проверим: {type} «{text}», {when}. Всё так? Озвучка: {Type}, {when} — верно?
confirm_no kind: clarify	Ответ «Нет» на подтверждение	1. Хорошо. Что поправить? 2. Понял. Что изменить: дату, время или текст? 3. Давайте исправим. Что не так?
conflict_one kind: conflict	На это время уже одна встреча (`{text}` — её текст)	1. На {time} уже есть встреча: «{text}». Что сделать? 2. Это время занято: «{text}». Как поступим? Озвучка: Это время занято. Что сделать?
conflict_many kind: conflict	На это время уже {count} встречи	1. На {time} уже {count:встреча|встречи|встреч}. Что сделать? 2. Это время занято: {count:встреча|встречи|встреч}. Как поступим? 3. {When} уже {count:встреча|встречи|встреч}. Что выберете? Озвучка: Это время занято. Что сделать?

`conflict_new_time` kind: clarify	Нажато «Другое время для новой»	1. На какое время поставить новую встречу? 2. Во сколько будет новая встреча? 3. Назовите время для новой встречи.
`conflict_move_existing` kind: clarify	Нажато «Перенести существующую» (`{text}` — существующей)	1. На какое время перенести «{text}»? 2. Во сколько теперь «{text}»? 3. Назовите новое время для «{text}».
`conflict_keep_both` kind: created	Нажато «Оставить обе». В `records` — новая встреча с `new`	1. Сохранил обе встречи. 2. Готово: обе встречи в календаре. 3. Оставил обе — в одно время.
`cancelled` kind: cancelled	Нажато «Отменить»	1. Отменил. 2. Хорошо, ничего не сохраняю. 3. Отменено.
`not_understood` kind: not_understood	Запрос не понят. Дальше — примеры кнопками	1. Боюсь, не понял. Можно, например, так: 2. Не уловил. Попробуйте так: 3. Не понял запрос. Вот примеры:
`off_topic` kind: off_topic	Запрос не по теме. Дальше — примеры кнопками	1. Это не по моей части. Я веду задачи, встречи и напоминания. 2. С этим не помогу, зато слежу за вашим расписанием. 3. Здесь я не советчик. А вот ваше время — моя забота. Озвучка: Это не по моей части.
`help` kind: help	«Что ты умеешь?». Дальше — примеры кнопками	1. Я Stevens. Записываю задачи, встречи и напоминания, нахожу их и подсказываю, что впереди. 2. Я Stevens, слежу за вашим временем. Скажите, например: «Напомни завтра в 10 позвонить Ивану». 3. Stevens, к вашим услугам. Запишу, найду и расскажу о планах — говорите или пишите. Озвучка: Я Стивенс: задачи, встречи, напоминания. v1.3 В `text` имя латиницей, в `speech` — кириллицей, чтобы синтез речи произнёс его правильно.
`thanks` kind: thanks	Пользователь благодарит	1. Рад помочь. 2. Пожалуйста. 3. К вашим услугам.
`error_backend` kind: error	Сбой на нашей стороне. «Повторить» фронт добавит сам	1. Извините, у меня сбой. Попробуйте ещё раз. 2. Прошу прощения, на моей стороне что-то пошло не так. Повторите, пожалуйста. 3. Извините, не получилось. Попробуйте через минуту. Озвучка: Извините, сбой. Повторите, пожалуйста.

Подписи кнопок, которые присылает бэк

Где	Кнопки (`label` → что отправляют)
`clarify` (любой)	При желании — быстрые ответы («10:00» → `send.text` «в 10:00»), всегда последней — «Отменить» → `cancel`, `secondary`
`confirm`	«Да, верно» → `confirm_yes` · «Нет» → `confirm_no`, `secondary`
`conflict`	«Другое время для новой» → `change_new_time` · «Перенести существующую» → `move_existing` (только если встреча одна и бэк умеет изменять записи) · «Оставить обе» → `keep_both` · «Отменить» → `cancel`, `secondary` (P-28, P-129)
`not_understood`, `off_topic`	Три примера (`send.text` = надпись): «Напомни завтра в 10 утра позвонить Ивану» · «Что у меня на следующей неделе?» · «Найди задачу про презентацию»
`help`	Те же три + «Поставь встречу с Иваном завтра»

10. Примеры ответов

Примеры показывают формат ответа. Данные взяты из CSV организаторов, даты в примерах посчитаны от «сегодня» = вс, 18 окт 2026. **Тестовые часы для проверки — ср, 21 окт 2026, 10:00 MSK (раздел 13)**: заглушки фронта посчитаны от них, поэтому даты и число записей там другие. При изменении формата обновляйте примеры. Тексты собраны по шаблонам раздела 9, у каждого указан ID и вариант.

created — «Напомни мне завтра в 10 утра позвонить Ивану» created_reminder, в. 1

```
{
  "message_id": "m-101", "kind": "created",
  "text": "Записал напоминание на завтра в 10:00: «Позвонить Ивану».",
  "speech": "Напоминание — завтра в 10:00.",
  "records": [{
    "id": 1001, "type": "reminder", "text": "Позвонить Ивану",
    "date": "2026-10-19", "time": "10:00", "date_kind": "exact",
    "status": "open", "created_at": "2026-10-18T14:05:10+03:00",
    "marks": ["new"], "highlight": null
  }],
  "nearest": null, "total": null, "awaiting_reply": false, "buttons": []
}
```

list, пусто — «Что у меня сегодня?» list_empty_nearest, в. 1

```
{
  "message_id": "m-102", "kind": "list",
  "text": "На сегодня планов нет. Ближайшее — вт, 20 окт в 18:00: «Обсудить бюджет по презентации».",
  "speech": "На сегодня планов нет. Ближайшее — 20 октября в 18:00.",
  "records": [],
  "nearest": {
    "id": 687, "type": "task", "text": "Обсудить бюджет по презентации",
    "date": "2026-10-20", "time": "18:00", "date_kind": "exact",
    "status": "open", "created_at": null, "marks": [], "highlight": null
  },
  "total": 0, "awaiting_reply": false, "buttons": []
}
```

found — «Когда у меня была встреча с Иваном?» found_when_last, в. 1

```
{
  "message_id": "m-103", "kind": "found",
  "text": "В последний раз — чт, 5 мар в 15:45: «Встреча с Иваном по презентации».",
  "speech": "В последний раз — 5 марта в 15:45.",
  "records": [
    { "id": 530, "type": "meeting", "text": "Встреча с Иваном по презентации",
      "date": "2026-03-05", "time": "15:45", "date_kind": "exact", "status": null,
      "created_at": null, "marks": [], "highlight": "date" },
    { "id": 429, "type": "meeting", "text": "Встреча с Иваном по презентации",
      "date": "2025-09-27", "time": "15:45", "date_kind": "exact", "status": null,
      "created_at": null, "marks": [], "highlight": "date" },
    { "id": 309, "type": "meeting", "text": "Встреча с Иваном по CRM",
      "date": "2025-04-04", "time": "18:00", "date_kind": "exact", "status": null,
      "created_at": null, "marks": [], "highlight": "date" }
  ],
  "nearest": null, "total": 3, "awaiting_reply": false, "buttons": []
}
```

clarify — «Поставь встречу с Иваном завтра» clarify_meeting_time, в. 1

```
{
  "message_id": "m-104", "kind": "clarify",
  "text": "Во сколько встреча?",
  "speech": null,
  "records": [], "nearest": null, "total": null, "awaiting_reply": true,
  "buttons": [
    { "label": "10:00", "send": { "text": "в 10:00" } },
    { "label": "15:00", "send": { "text": "в 15:00" } },
    { "label": "Отменить", "send": { "action": { "id": "cancel" } }, "style": "secondary" }
  ]
}
```

confirm — «Напомни про отчёт в пятницу в 7» confirm, в. 1

```
{
  "message_id": "m-105", "kind": "confirm",
  "text": "Правильно понимаю: напоминание «Отчёт», пт, 23 окт в 19:00?",
  "speech": "Напоминание, 23 октября в 19:00 — верно?",
  "records": [{
    "id": null, "type": "reminder", "text": "Отчёт",
    "date": "2026-10-23", "time": "19:00", "date_kind": "exact",
    "status": "open", "created_at": null, "marks": ["draft"], "highlight": null
  }],
  "nearest": null, "total": null, "awaiting_reply": true,
  "buttons": [
    { "label": "Да, верно", "send": { "action": { "id": "confirm_yes" } } },
    { "label": "Нет", "send": { "action": { "id": "confirm_no" } }, "style": "secondary" }
  ]
}
```

conflict — «Добавь встречу с Марией 21 октября в 8 утра» conflict_one, в. 1


```
{
  "message_id": "m-106", "kind": "conflict",
  "text": "На 08:00 уже есть встреча: «Встреча с Марией по презентации». Что сделать?",
  "speech": "Это время занято. Что сделать?",
  "records": [
    { "id": null, "type": "meeting", "text": "Встреча с Марией",
      "date": "2026-10-21", "time": "08:00", "date_kind": "exact", "status": null,
      "created_at": null, "marks": ["draft"], "highlight": null },
    { "id": 688, "type": "meeting", "text": "Встреча с Марией по презентации",
      "date": "2026-10-21", "time": "08:00", "date_kind": "exact", "status": null,
      "created_at": null, "marks": ["conflict"], "highlight": null }
  ],
  "nearest": null, "total": null, "awaiting_reply": true,
  "buttons": [
    { "label": "Другое время для новой", "send": { "action": { "id": "change_new_time" } } },
    { "label": "Перенести существующую",
      "send": { "action": { "id": "move_existing", "record_id": 688 } } },
    { "label": "Оставить обе", "send": { "action": { "id": "keep_both" } } },
    { "label": "Отменить", "send": { "action": { "id": "cancel" } }, "style": "secondary" }
  ]
}
```


thanks — «Спасибо!» thanks, в. 3


```
{
  "message_id": "m-107", "kind": "thanks",
  "text": "К вашим услугам.",
  "speech": null,
  "records": [], "nearest": null, "total": null, "awaiting_reply": false, "buttons": []
}
```


11. Задел под допы

Всё ниже в MVP не нужно, но формат уже оставляет место, поэтому фронт не придётся переделывать.

Доп	Что меняется в контракте
Д1 Изменение и перенос	`kind: "updated"`, действие `move_existing`
Д2 Удаление	`kind: "deleted"`, действие `delete`
Д3 Отметка выполненной	`kind: "completed"`, `status: "done"`, действие `complete`
Д4 Контекст диалога	Ничего: `conversation_id` уже есть
Д5 Повторяющиеся v1.3	Формат обсудим, когда дойдём до допов
Д6 Календарная панель	`GET /api/records?from=&to=`
Д7 Пуш-уведомления	Поле `remind_at` у записи
Д8 Несколько пользователей	v1.3 Перенесено в MVP — раздел 14
Д9 Несколько действий в запросе	Ничего: `records` уже массив
Д10 Отмена созданного	Кнопка с действием `undo` в ответе `created`
Д11 Свободное время при совпадении	Ничего: это дополнительные кнопки в `conflict`
Д12 Озвучка ответов	Ничего: поле `speech` уже есть
Распознавание на сервере	`POST /api/stt`: аудио → `{"text": "..."}`

12. Открытые вопросы и задачи

1. **Тип у записей из CSV** (O-01): угадываем по первому слову («Встреча», «Созвон», «Демо» → встреча, остальное → задача) или оставляем `null`? Фронт умеет оба варианта.
2. **Успеет ли бэк изменение записей в MVP?** От этого зависит кнопка «Перенести существующую».
3. **Границы периодов**: «эта неделя» — пн–вс? Что такое «вечером», «утром», «до конца недели»? Решает бэк, но ответ стоит записать в README.
4. **Скорость LLM**: укладываемся ли в 25 секунд на запрос?
5. **Переменные шаблонов** (раздел 9): удобен ли такой набор для вашего шаблонизатора? Если чего-то не хватает — скажите, допишем.
6. **Успеете ли вход к сдаче?** v1.3 Если нет — не страшно: фронт выключит экран входа флагом и откроется сразу в песочнице. Но песочница по `X-Client-Id` нужна в любом случае. Скажите заранее.

> **Задача бэку: проверить хостинг из России** v1.3 (P-168)
>
> Жюри откроет ссылку из России. По статьям 2026 года Vercel, Netlify и Cloudflare Pages оттуда открываются нестабильно, особенно с мобильного интернета; у GitHub с мая заметная доля сбоев. Если ссылка не откроется, остальное не важно.
>
> 1. Залить страницу-заглушку на 3 хостинга статики: GitHub Pages, Vercel, Яндекс Cloud (Object Storage). Для Яндекс Cloud проверить: нужна ли привязка карты для бесплатного уровня и работает ли HTTPS без своего домена (без HTTPS не будет микрофона).
> 2. То же — для хостинга бэка, который вы выбрали: простой `GET /api/health`.
> 3. Открыть каждую ссылку с телефонов команды на разных операторах (МТС, Билайн, МегаФон, Т2) и с домашнего Wi-Fi. Записать: открылось / долго / не открылось.
> 4. Выбрать основной хостинг фронта и запасной — на другом сервисе (запасной деплой с минимальным набором функций, P-149).

13. Тестовые фразы

Основано на сценариях аналитика команды. Полная версия — в файле `golden_dataset_v2.json`: для каждой фразы там ожидаемый разбор LLM (`expected_parse`) и ожидаемый ответ бэка с конкретными датами (`expected_response`) (P-137).

- **Тестовые часы:** `client_time = 2026-10-21T10:00:00+03:00` (среда). От них считаются все относительные даты, поэтому результаты воспроизводимы (P-138).
- **ИИ-модуль** сравнивает выход LLM с `expected_parse`. **Бэкенд** прогоняет фразы через `POST /api/chat` и сравнивает с `expected_response`. Схема полей — в `meta` файла.

№	Фраза	`kind`	Шаблон
1	Напомни завтра в 10 подготовить отчёт	`created`	**`created_reminder`**
2	Поставь задачу на пятницу: отправить договор Ивану	`created`	**`created_task`**
3	Напомни послезавтра в 9 проверить смету	`created`	**`created_reminder`**
4	Поставь задачу на понедельник обновить материалы по CRM	`created`	**`created_task`**
5	Напомни сегодня в 18 позвонить Сергею	`created`	**`created_reminder`**
6	Созвон с Марией в среду в 15:00	`created`	**`created_meeting`**
7	Встреча с командой послезавтра в 11	`created`	**`created_meeting`**
8	Поставь встречу с Натальей по CRM на завтра в 12	`created`	**`created_meeting`**

9	Запланируй планёрку с Альфой по бюджету в понедельник в 10	`created`	**`created_meeting`**
10	Демо проекта в четверг в 16:00	`created`	**`created_meeting`**
11	Поставь встречу с Иваном завтра	`clarify`	**`clarify_meeting_time`**
12	Напомни про отчёт	`clarify`	**`clarify_reminder_when`**
13	Созвон с партнёром в 14:00	`clarify`	**`clarify_day`**
14	Поставь встречу на пятницу в 15	`clarify`	**`clarify_meeting_person`**
15	Напомни на следующей неделе согласовать презентацию	`clarify`	**`clarify_reminder_when`**
16	Что у меня сегодня?	`list`	**`list_some`**
17	Какие планы на сегодня?	`list`	**`list_some`**
18	Покажи мои встречи на завтра	`list`	**`list_type_empty`**
19	Найди задачу по договору	`found`	**`found_some`**
20	Когда у меня встреча с Марией?	`found`	**`found_when_next`**
21	Что у меня было в прошлую пятницу?	`list`	**`list_some`**
22	Поставь задачу купить билеты до конца недели	`created`	**`created_task_deadline`**
23	В пятницу нужно отправить отчёт	`created`	**`created_task`**
24	Запиши задачу разобрать почту	`created`	**`created_task_nodate`**
25	В четверг в 11 у меня встреча с дизайнером	`created`	**`created_meeting`**
26	Напомни вчера в 10 позвонить Анне	`clarify`	**`clarify_date_past`**
27	Поставь встречу с Еленой в субботу в 20:45	`conflict`	**`conflict_one`**
28	Напомни про отчёт в пятницу в 7	`confirm`	**`confirm`**
29	Есть ли у меня встречи сегодня?	`list`	**`list_type_yes`**
30	Покажи мои встречи на сегодня	`list`	**`list_some`**
31	Что запланировано на эту неделю?	`list`	**`list_some`**
32	Что у меня 6 ноября?	`list`	**`list_empty_nearest`**
33	Я когда-то ставил задачу по договору, найди её	`found`	**`found_some`**
34	Когда у меня была встреча с Иваном?	`found`	**`found_when_last`**
35	Какая завтра погода?	`off_topic`	**`off_topic`**
36	Что ты умеешь?	`help`	**`help`**
37	Спасибо, Стивенс	`thanks`	**`thanks`**
38	Сделай что-нибудь с этим	`not_understood`	**`not_understood`**
39	Поставь встречу с Иваном завтра → в 15 (диалог)	`clarify` → `created`	**`clarify_meeting_time`** → **`created_meeting`**
40	Поставь встречу с Еленой в субботу в 20:45 → [кнопка] Оставить обе (диалог)	`conflict` → `created`	**`conflict_one`** → **`conflict_keep_both`**
41	Напомни про отчёт → Что у меня сегодня? (диалог)	`clarify` → `list`	**`clarify_reminder_when`** → **`list_some`**

14. Песочница и аккаунты v1.3

Авторизация перешла из допов в MVP (P-179). Цель — чтобы каждый, кто открыл ссылку, работал со своими данными, а жюри попадало внутрь за один клик.

Два режима

Режим	Как попадают	Данные
Песочница	Кнопка «Попробовать без регистрации»	Своя копия демо-данных из CSV организаторов (1000 записей) на устройство. Изменения сохраняются между заходами. Сброс — `POST /api/sandbox/reset` (P-181, P-187)
Аккаунт	Регистрация или вход по логину и паролю, без почты (P-180)	Свои записи. При регистрации аккаунт **пустой**, даже если человек до этого работал в песочнице: песочница не переносится (P-181). Вход с другого устройства — те же записи аккаунта, без слияния с песочницей (P-182)

Заголовок режима

- **Песочница:** `X-Client-Id: <uuid>`. Фронт создаёт UUID при первом открытии и хранит в браузере (P-164). Неизвестный id → бэк заводит песочницу и копирует в неё демо-данные. Дальше все запросы с этим id работают с этой копией.
- **Аккаунт:** `Authorization: Bearer <token>` (P-184).
- Пришли оба — используется `Authorization`. Не пришло ни одного — общая демо-база (на случай старого клиента).
- Нужен для `/api/chat`, `/api/records`, `/api/auth/logout`, `/api/auth/me`, `/api/sandbox/reset`. Не нужен для `/api/health`, `/api/auth/register`, `/api/auth/login`.
- `conversation_id` уникален в рамках одного режима и пользователя.

Правила логина и пароля

- **Логин:** 3–32 символа — русские и латинские буквы, цифры, точка, дефис, подчёркивание: `^[A-Za-zA-Яа-яЁё0-9._-]{3,32}$`. Пробелы по краям обрезаем. Уникален **без учёта регистра** (храним как ввели, сравниваем в нижнем регистре) (P-189).
- **Пароль:** не короче 5 символов, не длиннее 128. Пробелы не обрезаем. Храним только хэшем (bcrypt или argon2), никогда открытым текстом.
- **Токен:** случайная строка или JWT — на ваш выбор. Живёт не меньше 30 дней; после `logout` перестаёт действовать.
- Фронт проверяет те же правила до отправки, но бэк проверяет обязательно.

Запросы и ответы

Запрос	Успех	Ошибки
`POST /api/auth/register` `{"login": "matvey", "password": "12345"}`	`200` `{"token": "…", "user": {"id": 7, "login": "matvey"}}`	`409 {"error": "login_taken"}` `422 {"error": "invalid_login"}` `422 {"error": "invalid_password"}`

`POST /api/auth/login` то же тело	`200`, как у регистрации	`401 {"error": "invalid_credentials"}` — и при неверном пароле, и при несуществующем логине (не подсказываем, что именно не так)
`POST /api/auth/logout`	`204`	—
`GET /api/auth/me`	`200 {"user": {"id": 7, "login": "matvey"}}`	`401`
`POST /api/sandbox/reset`	`204`, демо-данные песочницы восстановлены	`422`, если нет `X-Client-Id`
Любой запрос с недействительным токеном	—	`401` → фронт выводит на экран входа с «Сессия закончилась»

Что видит пользователь

- Первый экран — форма входа и регистрации, под ней «Попробовать без регистрации» с пояснением: «Откроется песочница с демо-данными организаторов Gorkycode — 1000 готовых планов. Ваши изменения сохранятся на этом устройстве.» (P-185, P-186)
- Выбор запоминается: при следующем открытии — сразу в приложение (P-188).
- В шапке видно режим: метка «Песочница» с меню (сбросить демо-данные, войти) или кружок с первой буквой логина с меню (логин, выйти) (P-190).
- Если бэк не успеет вход — фронт выключит его флагом и откроется сразу в песочнице. Тогда из этого раздела нужны только `X-Client-Id` и `/api/sandbox/reset`.